

発表スクリプト — IntSeqBERT @ CICM 2026

- 英語表現の目安: B2 下位。専門用語は残し、短い文と直接的な説明を使う。Tao 氏の引用部分は原文を維持する。
- slides.tex の本編 20 枚・予備 3 枚と同じ順序。スライド本文の目安は B2 中位。
- 各スライドでは、太字の完結した文を上から読むと説明がつながる。数値・用語だけの太字は、参照用の強調。
- 「スライドを拾い読みする場合」はスライド上の文と一致する。「補足説明」は必要な箇所だけ加え、両方を続けて全て読む必要はない。
- > の段落と Bridge は読み上げ候補。日本語メモは読み上げない。
- 発表 25 分＋質疑 5 分を想定。英文は説明用の候補文として残し、当日の流れに応じて省略・補足する。固定のスライド別時間割は設けない。
- スライドの First / Next / However / So などそのまま読める。Bridge は、さらに一言つなぎたいときだけ使う。
- 本編の候補文の語数は末尾参照。予備スライドは質疑用で、本編時間に含めない。

Slide 1: Title

Good morning, everyone. I am Kazuhisa Nakasho, from Iwate Prefectural University in Japan. Today I will talk about IntSeqBERT. It is a Transformer encoder for integer sequences in the OEIS. We want to learn the arithmetic structure of these sequences. Our code and data are public on GitHub.

Bridge: “Let me start with a recent comment by Terence Tao.”

Slide 2: Why Finding Good Problems Matters

原文読み上げ（Tao 氏の投稿 1/4）：

I wrote recently about how the collection of **good, fruitful open problems** is now being mined in a non-renewable fashion, leading to the potential scenario of **these problems becoming scarce**. This may seem unintuitive at first, since the set of possible problems one could ask is infinite. Perhaps the following analogy can help: a country or region can suffer a **critical shortage of drinking water** while simultaneously being **surrounded by a massive ocean**.

One can easily generate any number of open problems in mathematics at will, such as working out the $10^{10^{10}}$ th digit of pi. But the vast majority of such problems are not worth focusing attention on: they show no particular propensity to **reveal any further insights or connections to other questions**, or may either be too easy or too impossible relative to known techniques to learn anything from the exercise. (1/4)

短い解説（本研究とのつながり）：

We can ask many questions, but useful questions help us understand more mathematics. This motivates our long-term goal: to help find new mathematical connections and promising conjectures. As a first step, we study how AI can learn arithmetic patterns in integer sequences.

Bridge: “Let me show how numerical patterns can lead to new mathematics.”

発表メモ: 太字は読み上げのポイントで、スライドの強調箇所と対応する。強調は発表者によるもの。まずスライド上の引用を原文のまま読み、その後に短く解説する。引用は B2 に言い換えず、解説のみ B2 下位を目安にする。「I」は Tao 氏を指す。 $10^{10^{10}}$ th digit of pi は “the ten to the power of ten to the power of ten-th digit of pi” と読む（指数は右結合）。末尾の (1/4) は投稿

番号なので読み上げない。引用後の本研究の動機は発表者自身の解説。

出典: Terence Tao, Mathstodon, post 1/4。ご提示の本文に基づく。投稿日は未確認のため記載しない。

Slide 3: Long-Term Goal — AI for Mathematical Discovery

スライドを拾い読みする場合（太字の文を上から読む）：

New mathematical connections often begin with numerical patterns. Our long-term goal is to find similar arithmetic patterns across sequences and use them to suggest new conjectures. As a first step, we build a representation that captures arithmetic structure.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

New connections have often led to important conjectures. McKay noticed that 196,884 is 196,883 plus one. The first number appears in the j -function. The second is a dimension related to the Monster group. This observation led to Monstrous Moonshine. The BSD conjecture and mirror symmetry also began with numerical patterns.

Today, AI systems can take part in mathematical discovery. Examples include AlphaProof, the Ramanujan Machine, and FunSearch. Our long-term goal is to find similar arithmetic patterns in sequences from different fields. These connections could help us make new conjectures.

We use about 392,000 OEIS sequences, each with a mathematical definition. To compare them, we need a representation that a computer can process. In this paper, we build one and measure how much arithmetic structure it learns.

Bridge: “There is one main difficulty: learning multiplication.”

Slide 4: Why Multiplication Is Hard for Deep Learning

スライドを拾い読みする場合（太字の文を上から読む）：

However, standard neural models struggle to learn multiplication rules. Tokenisation also hides arithmetic structure.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

Standard neural models have difficulty learning multiplication rules. One reason is spectral bias. Networks learn low-frequency patterns, which are more like addition, first.

This table separates four results by their model and task. In the first row, a threshold circuit with three layers can multiply n -bit numbers. Three layers are enough, and fewer layers are not enough in that circuit setting. But the existence of a small circuit does not mean that training will find it.

The second row shows a practical difficulty. In the cited study, GPT-4 gets 59 percent of three-digit multiplication questions correct. For four-digit multiplication, this falls to only 4 percent.

The third row is about multiplication modulo a prime. A two-layer Transformer learns the general rule only after a very long training period. This delayed learning is called grokking.

The fourth row shows a successful method. A twelve-layer model can multiply fifteen-digit numbers with special implicit chain-of-thought training. These rows use different models and tasks. The layer counts cannot be

compared directly.

Tokenisation creates another problem. Values outside the vocabulary all become UNK. Token IDs also do not directly show arithmetic structure. The OEIS makes this problem important: its values range from single digits to around ten to the power of 210.

Bridge: “Our idea is to give the model useful information from number theory.”

発表メモ: 表の行に沿って必要な結果を説明する。3 層の回路理論と、2 層・12 層の学習実験は前提が異なる。「どの層数が正しいか」という比較ではなく、理論上の表現能力と学習で得られる性能を分けて説明する。元の論点・数値・出典は全て本編に残す。

Slide 5: Key Idea — Show Multiplicative Structure in the Input

スライドを拾い読みする場合（太字の文を上から読む）:

So we give the model residues, which preserve addition and multiplication. The residues stay small, even when the original values are very large. Our hypothesis is that these inputs make multiplication easier to learn.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）:

We use modular arithmetic. The residue of x modulo m is the remainder when we divide x by m . This operation is a ring homomorphism. This means that it preserves both addition and multiplication.

The residues keep information about multiplication, even when the original numbers are very large. For a number as large as ten to the power of 210, the residue is still between zero and m minus one.

Number theory also explains periodic patterns in these residues. Fermat’s little theorem describes how powers repeat modulo a prime. Quadratic reciprocity connects residue information across primes. The Chinese Remainder Theorem, or CRT, combines information from different prime powers.

This leads to our hypothesis. We give the network residues for many moduli. We call this the modulo spectrum. Then information about multiplication is available in the input layer, at depth zero. The model does not have to learn all this structure from the original values alone.

Bridge: “Now let me explain the task we use to test this idea.”

Slide 6: Task — Masked Sequence Modelling on OEIS

スライドを拾い読みする場合（太字の文を上から読む）:

To test this idea, we hide sequence terms and ask the model to predict them. This tests whether the model has learned the rule of the sequence. For each hidden term, we predict its magnitude, sign, and residues.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）:

Our task is masked sequence modelling. It is like BERT, but with integers. We take up to the first 128 terms of a sequence and hide 15 percent of the positions. The model predicts the hidden values from the other terms. This tests how well it has learned the rule behind the sequence. If we hide only the last term, the task becomes next-term prediction.

An earlier benchmark, FACT, tested a standard Transformer on five OEIS tasks. Predicting masked terms was the hardest task. Our Vanilla baseline uses that Transformer architecture.

For each hidden position, our model predicts three things. First, the magnitude, or size, on a log scale. Second, the sign: positive, negative, or zero. Third, the residues for every modulus from 2 to 101.

Bridge: “The model uses two input streams to make these predictions.”

Slide 7: IntSeqBERT — Dual-Stream Architecture

スライドを拾い読みする場合（太字の文を上から読む）：

The model uses two input streams: magnitude and modulo. We combine the streams and pass the result to a Transformer.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

This figure shows IntSeqBERT. The first stream is the magnitude stream. It represents the log-scale size and the sign using four numbers.

The second stream is the modulo stream. For each modulus from 2 to 101, we put the residue on a unit circle using sine and cosine. This gives us 200 numbers in total. It uses the idea of sinusoidal positional encoding, but here we encode residues instead of positions. On the circle, residue m minus one is close to residue zero. This matches the cyclic structure of modular arithmetic.

We combine the two streams with FiLM. The modulo stream gives a scale and a shift. We apply them to each element of the magnitude stream. The result goes into a standard pre-LN Transformer encoder. We test three model sizes. The largest has 91.5 million parameters.

Bridge: “We train this encoder with three prediction tasks.”

発表メモ：図を左から右へ指す。FiLM の数式全体は読み上げず、scale と shift を説明する。

Slide 8: Training — Multi-Task Objective

スライドを拾い読みする場合（太字の文を上から読む）：

We train the model to predict all three quantities together. All experiments run on a single consumer GPU.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

The encoder has three prediction heads: magnitude, sign, and modulo. The magnitude head also predicts uncertainty. The Solver will use this later. The modulo head has one classifier for each of the 100 moduli. We normalise each loss by its maximum entropy. This keeps the losses on a similar scale. We add the three losses, with a weight of two for the modulo loss.

We train on 219,765 sequences for 200 epochs. We use FP32 because very large values cause overflow problems with FP16. All experiments use one RTX 3070 Ti GPU with eight gigabytes of memory. Other researchers can repeat the experiments on similar hardware.

Bridge: “Next, we need to turn the model’s predictions into integers.”

Slide 9: Neural-Symbolic Decoding — the CRT Solver

スライドを拾い読みする場合（太字の文を上から読む）：

First, the Solver generates candidates in the predicted magnitude range. Next, it scores candidates using magnitude and all 100 modulo distributions. We add weighted log-probabilities. Finally, it returns the candidates with the highest scores.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

The Solver combines the model's probability distributions to rank integer candidates. First, magnitude and uncertainty give a three-sigma search interval. For a small interval, we check every integer. For larger intervals, CRT search generates candidates from likely residue patterns. The three search modes are shown on the slide.

Now we score each positive candidate n . For every modulus from 2 to 101, we calculate n modulo m . We look up the predicted probability of that residue. We add the log-probabilities from all 100 moduli. Then we multiply this sum by 0.3 and add the magnitude log-likelihood. The weight reduces the effect of shared information, for example between mod 2 and mod 4.

This is the probabilistic part of our CRT Solver. A good candidate needs support from both its magnitude and its full residue pattern. We return the candidates with the highest total scores. The predicted sign is applied separately. If the sign is zero, we return zero directly.

Bridge: “Before the results, let us look at the data and the evaluation measures.”

発表メモ：確率を直接足すのではなく、候補に対応する対数確率を全 modulus にわたり加算する。magnitude 項は候補間で共通の定数を除いた Gaussian 対数尤度。候補生成で使う moduli と、最終採点に使う全 100 個の moduli を区別する。

Slide 10: Dataset — 274,705 OEIS Sequences

スライドを拾い読みする場合（太字の文を上から読む）：

We keep about 70 percent of the OEIS entries for this task. We exclude eleven tags, grouped by the reason for exclusion. We also group values by magnitude to compare easier and harder cases.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

We start with 391,710 OEIS entries and keep 274,705, or about 70 percent. We split them into training, validation, and test sets in a ratio of eight to one to one.

We exclude eleven tags in three groups. The first group contains sequences whose patterns depend on how values are written. Examples include decimal expansions and continued fractions. The second group does not fit our task setting. Tables are one example. The third group contains quality labels.

These choices are open to discussion. For example, the labels less and dumb depend on people's judgement. Tables could also be converted into sequences. Future benchmarks may make different choices. For evaluation, we divide values into five size groups, which we call value buckets, or magnitude buckets. They range from Small, below 100, to Astronomical, at least ten to the power of fifty.

Bridge: “One measure needs a short explanation.”

Slide 11: Metrics — and Three Questions

スライドを拾い読みする場合（太字の文を上から読む）：

We measure size, residues, and exact integer prediction. We ask three questions: does the dual stream help, where does it help, and can we predict exact integers?

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

This table shows what each metric measures. Magnitude accuracy, or Acc zero point five, checks the approximate size of the value. It counts predictions within zero point five of the true magnitude on the log scale. This helps us judge the magnitude prediction used for the Solver's search range.

MMA is the average classification accuracy across all 100 moduli. NIG means normalised information gain. It measures learning for each modulus. Zero means no information gain over a uniform prediction. One means perfect prediction.

Solver Top-k measures exact integer prediction. It counts a prediction as correct if the true integer is among the top k candidates.

The example below the table shows why magnitude accuracy is different from exact accuracy. A difference of zero point five on the log scale is a factor of about 3.16. If the true value is one million, predictions from about 320,000 to 3.2 million count as correct.

We ask three questions about the results. Does the dual stream help? Where does it help? And can we predict exact integers?

Bridge: “Let us start with the first question.”

発表メモ：表を上から指し、各指標の役割を説明する。具体例は magnitude accuracy のみの判定例であり、整数の厳密一致ではない。

Slide 12: Q1 — Does It Help? Yes, Consistently

スライドを拾い読みする場合（太字の文を上から読む）：

First, the dual stream improves both magnitude and residue prediction. Removing the modulo stream makes both scores worse. The same pattern holds for the Small and Middle models.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

Yes, the dual stream helps. For our largest model, magnitude accuracy is 95.85 percent. This is 8.9 percentage points above the Vanilla baseline. Mean modulo accuracy is 50.38 percent, which is 4.5 points above Vanilla.

Now look at the ablation row. This is the model without the modulo stream. Removing that stream reduces mean modulo accuracy by 15.2 points. It also reduces magnitude accuracy by 6.2 points. So the modulo stream helps with both tasks. The residues limit which integer values are possible. This also helps the model predict magnitude.

We see the same pattern for the Small and Middle models. The next slide shows how the results change with model size.

Bridge: “Let us now compare all three model sizes.”

Slide 13: Full Results & Scaling Behaviour

スライドを拾い読みする場合（太字の文を上から読む）：

IntSeqBERT has the highest modulo accuracy at every model size. Larger models help residue prediction more than magnitude prediction.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

These three panels compare our Small, Middle, and Large models. Here, size means model size, not the size of the integer values. The panels show magnitude accuracy, mean modulo accuracy, and exact prediction accuracy.

For mean modulo accuracy, the ranking stays the same at all three sizes. IntSeqBERT comes first, followed by Vanilla and then the ablation model. For IntSeqBERT, going from Small to Large adds ten percentage points to modulo accuracy. Magnitude accuracy rises by only 1.1 points. So larger models help modulo prediction more than magnitude prediction.

The right panel shows Solver Top-1 accuracy. It rises by about five points, from 14.05 to 19.09 percent. We will look at exact prediction in more detail shortly.

At epoch 200, the training loss is 1.00 and the validation loss is 1.01. These results show no clear sign of overfitting. We plan to train at a larger scale.

Bridge: “The second question is where the model helps most. Let us group the results by integer size.”

発表メモ：左図（magnitude）→中央図（modulo）→右図（Solver）の順で示す。改善幅は IntSeqBERT の Small → Large。MMA のモデル順位と、各指標の改善幅を区別する。

Slide 14: Q2 — Where? Exactly Where Tokenisation Breaks

スライドを拾い読みする場合（太字の文を上から読む）：

Next, the clearest gains are in the Large value bucket and above. Overall scores can hide these differences.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

This plot compares predicted magnitude with true magnitude. Here, the value buckets group integers by magnitude. Look at the Vanilla model. Above its vocabulary limit, all values become UNK, and its predictions become less accurate. In the Large value bucket, its error is about thirteen times our model’s error. The overall R-squared values are 0.988 for IntSeqBERT and 0.943 for Vanilla.

We need to be careful with overall scores. Most OEIS sequences grow slowly, so easy cases have a large effect on these scores. That is why we also report results for each size group. The clearest gains appear in the Large value bucket and above. Our magnitude predictions are still useful there, while the baseline has difficulty.

Bridge: “Now for the third question: can we predict the exact integer?”

Slide 15: Q3 — The Strictest Test: Exact Next-Term Prediction

スライドを拾い読みする場合（太字の文を上から読む）：

Finally, we test whether the full system predicts the exact next term. IntSeqBERT reaches 19.09 percent, compared with 2.59 percent for Vanilla. However, our exact accuracy also falls to near zero for large values.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

We test the full system on 10,000 sequences kept separate for testing. The task is to predict the exact last term. Vanilla gets 2.59 percent correct with its first candidate. IntSeqBERT gets 19.09 percent. That is about 7.4 times the baseline score. With ten candidates, our score rises to 26.23 percent.

For small values, our Top-1 score is 68.3 percent. For medium values, it is 20.8 percent. Vanilla scores zero beyond its vocabulary. But our model also scores close to zero for the Large value bucket and above.

For large integers, the Solver must combine evidence from many moduli. Our average modulo accuracy is about 50 percent. Combining these uncertain predictions to find the correct integer remains difficult.

Bridge: “What does 19 percent mean in practice?”

Slide 16: What Does 19% Mean? — Better, but Still Low

スライドを拾い読みする場合（太字の文を上から読む）：

So the improvement is large, but the absolute accuracy is still low. Only about one in five predictions is exactly correct. The modulo stream helps, but exact prediction of OEIS terms remains difficult.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

The result shows clear progress, but the accuracy is still low. Vanilla gets 2.59 percent correct. IntSeqBERT gets 19.09 percent. That is about 7.4 times the baseline score.

But 19 percent means only about one in five predictions is exactly correct. Most predictions are still wrong. For large values, exact prediction is still close to zero.

So our message is simple. The modulo stream helps, but exact prediction of OEIS terms remains difficult.

Bridge: “Next, let us look more closely at the arithmetic patterns the model learned.”

Slide 17: But What Did the Network Learn? The NIG Spectrum

スライドを拾い読みする場合（太字の文を上から読む）：

We also ask what arithmetic structure the model has learned. IntSeqBERT learns more at every modulus, with a clear pattern of peaks. The highest peak is at modulus 96, followed by modulus 2.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

This plot shows NIG for every modulus from 2 to 101. IntSeqBERT scores above both baselines across the whole spectrum. But the scores are not the same for every modulus. The peaks follow a clear pattern.

Across all models and sizes, the best modulus is 96. This is two to the power of five, times three. The second best is 2, which tells us whether a number is even or odd. We call this parity. Parity patterns are common in the OEIS. For modulus 2, accuracy is 85.7 percent for IntSeqBERT, 81.4 for Vanilla, and 72.1 without the modulo stream.

Bridge: “Why does 96 work so well? The next plot helps explain this.”

Slide 18: Finding — Higher NIG with a Lower Totient Ratio

スライドを拾い読みする場合（太字の文を上から読む）：

We find higher NIG with a lower totient ratio. One modulus can combine information from several smaller moduli. This suggests testing more composite moduli.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

First, the observation. We compare NIG with Euler’s totient ratio, $\phi(m)$ divided by m . There is a strong negative correlation: minus 0.851. The p-value is below ten to the power of minus 28. In general, a lower totient ratio goes with a higher NIG score. This is an experimental result, not a theorem.

Next, our interpretation. We explain this using CRT. If we know x modulo 96, we also know x modulo 2, 4, 8, 16, and 32, as well as modulo 3. So one modulus gives information about several smaller moduli. Composite moduli with many small prime factors combine this information.

Finally, the next steps. We should test highly composite numbers when we extend the spectrum beyond 101. These are numbers with more divisors than any smaller positive integer. They may be more useful than adding more primes.

The spectrum also describes the arithmetic patterns in a dataset. We could use these spectra to compare sequences or datasets.

Bridge: “Comparing sequences brings us back to our original goal.”

発表メモ：Observation は実測結果、Interpretation はその解釈、Next steps は今後の提案。3つの見出しを順に指しながら説明する。

Slide 19: Positioning in Computer Mathematics

スライドを拾い読みする場合（太字の文を上から読む）：

This brings us back to our goal of finding mathematical connections. Our representation can support program synthesis by finding shared arithmetic structure. These similarities could help us suggest new conjectures.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

How could this work help computer mathematics? Program synthesis methods, such as Alien Coding, QSynt, and Learning Conjecturing, find rules that generate sequences. Our method learns arithmetic patterns shared

by many sequences. These two approaches could work together.

Program synthesis asks, what program generates this sequence? We ask, which sequences have similar number-theoretic structure? Their rules may look very different, but their arithmetic patterns may be similar.

Our learned representations could guide the search for a program. They could also help us find similar sequences in a large collection. These are possible future uses. Finding such connections could help us generate new conjectures. This is the long-term goal I described at the start.

Bridge: “Let me finish with four main points.”

Slide 20: Conclusion

スライドを拾い読みする場合（太字の文を上から読む）：

First, the modulo spectrum makes arithmetic structure visible in the input. Second, it improves prediction at every model size. Third, composite moduli combine useful arithmetic information. However, exact prediction remains difficult. Next, we want to improve decoding and use similar sequences to suggest conjectures. Thank you for listening.

補足説明（必要な箇所だけ、対応するスライドの文の後に追加）：

First, the modulo spectrum uses features based on number theory. A ring homomorphism makes information about multiplication available in the input, at depth zero.

Second, the model improves results at every size. Exact next-term prediction is about 7.4 times better than the baseline.

Third, composite moduli combine arithmetic information. We find higher NIG scores with lower totient ratios.

Fourth, exact prediction of OEIS terms is still hard. This supports further work on neural-symbolic methods.

Our next steps are to test approximate CRT for large integers, add highly composite moduli, and train larger models. We also want to use similar sequences to help generate conjectures. Our code and data are on GitHub. Thank you for listening.

Backup Scripts (質疑用・本編 25 分には含めない)

Backup 1: Why Not Digit-Based Tokenisation?

スライドを拾い読みする場合 (太字の文を上から読む) :

Another option is to generate integers one decimal digit at a time. Our input gives the model residues directly. Digit-by-digit generation also changes the prediction task.

補足説明 (必要な箇所だけ、対応するスライドの文の後に追加) :

Another option is to use one token for each decimal digit. We have not compared this approach with our method in this paper. Our input gives the model residues directly. With digit tokens, the model would need to calculate the residues from the digits. The prediction task also changes. It would generate a sequence of digits instead of predicting one integer value. We plan to compare the two approaches under the same conditions.

Backup 2: Limitations

スライドを拾い読みする場合 (太字の文を上から読む) :

Large integers remain difficult because residue predictions are uncertain. The dataset also affects our overall scores. Our experiments use limited compute and one random seed.

補足説明 (必要な箇所だけ、対応するスライドの文の後に追加) :

Large integers remain difficult because the Solver must combine uncertain predictions from many moduli. In 13.4 percent of Solver calls, no valid candidate is found. Approximate CRT or less strict residue conditions may help.

The dataset also affects the results. Many sequences contain only non-negative values, which helps sign accuracy. Many grow slowly, which helps overall magnitude accuracy.

We used one eight-gigabyte GPU and one random seed, 42. The baseline vocabulary has 20,003 tokens. Vanilla uses the same architecture as FACT, but with our training settings. We do not reproduce the numbers reported in the FACT paper.

Backup 3: How Hard Is Multiplication for Neural Networks?

スライドを拾い読みする場合 (太字の文を上から読む) :

The answer depends on the circuit type and the training method. Experiments show that learning multiplication can still be difficult.

補足説明 (必要な箇所だけ、対応するスライドの文の後に追加) :

The answer depends on the circuit type and the training method. In circuit theory, n -bit multiplication is not in AC zero, the class of constant-depth AND and OR circuits. But threshold circuits can do it with three layers and polynomial size. Fewer layers are not enough in that setting. Iterated products can be computed with four layers.

There is also a connection between Transformers and circuit complexity. The cited result gives uniform TC zero as an upper bound for log-precision Transformers.

The experiments on this slide show difficulties in learning multiplication. In the cited GPT-4 study, zero-shot accuracy falls from 59 percent for three-digit multiplication to 4 percent for four-digit multiplication. In the grokking study, a two-layer Transformer learns to generalise only after a very long training period. The final study reports failures with standard training even at 90 billion parameters. A twelve-layer model succeeds on fifteen-digit products with special implicit chain-of-thought training. Its attention forms a binary tree of partial products.

発表メモ: 専門用語の $AC^0 \cdot TC^0$ 等は維持。引用文献の詳細は予備スライド 3 を示す。

練習メモ

- 冒頭は Tao 氏の問題提起→歴史的な数値観察→本研究の長期目標の順。説明に使える情報を残し、必要に応じて読み上げを省略する。
 - 本編は Slide 1~20。Slide 9 は候補生成→確率の合算→順位付け、Slide 16 は改善幅と絶対的な正解率を説明する。
 - 結果は Q1 →モデルサイズ別の結果→ Q2 → Q3 →限界の解釈→ NIG →数論的な分析の順。
 - percentage points は割合の差、times は倍率として読み分ける。
 - modulus は単数、moduli は複数。初出の residue と parity は原稿内で説明する。
 - 図・数式・引用文献の細部は必要に応じて指し示す。全てを読み上げる必要はない。
 - 基本は太字の完結した文を上から拾い読みする。説明を足す場合は、同じ項目の通常の文や表の行を選ぶ。
 - 図のスライドも太字の文から読める。数式や表の全てを英語の文に直す必要はない。
 - 時間が押した場合は補足説明を省略する。聴衆が参照できる情報はスライドに残す。
-
- 拾い読み部分の語数: 約 760 語 (Slide 1 の導入、Slide 2 の原文引用と解説、Slide 3~20 の太字文。Bridge と補足は除く)。
 - 本編読み上げ語数: 約 3,345 語 (拾い読み文・補足・Bridge を合算した候補文の総量)。補足には拾い読み文と重なる説明があり、全てを続けて読むことは想定しない。